

AI - Lab 2: Deterministic Planning

Francesco Fuggitti
fuggitti@diag.uniroma1.it

AI Planning

3. PDDL

How to Explain Your Problems to a Computer

Jörg Hoffmann



Winter Term 2019/2020

PDDL

What is PDDL?

- Once you decided for STRIPS/FDR/whatever, you still need to design an input syntax that your computer can read.
- That input syntax in the planning area is PDDL: The [Planning Domain Definition Language](#).
- In particular, PDDL is used in the [International Planning Competitions \(IPC\)](#).

Why PDDL? It's just a fact of life:

→ PDDL is the de-facto standard input language in the planning area.

→ To complete this course (and for doing a BSc/MSc/PhD in the FAI group) you must know this language.

(When I started to work in planning, everybody used their own input language = needing an interpreter every time you talk to your neighbor.)

Our Agenda for This Chapter

- ② **Schematic Encodings:** Explains the main design principle behind PDDL.
- ③ **PDDL Grammar:** Outlines the syntax, with example snippets.
- ④ **History and Extensions:** Summary of what's out there and how we got there. (I'll skip this and leave it for you to read at home; and no, it's not exam-relevant.)

Schematic Encodings

Schematic encodings use variables that range over objects:

- **Predicates** instead of STRIPS propositions. **Arity**: number of vars.
- **Action schemas** instead of STRIPS actions. **Arity**: number of vars.
- Analogy: propositional logic vs. predicate logic (PL1).
- **Set of objects in PDDL is finite!**

→ Like predicate logic, PDDL describes the world in a schematic way relative to a set of objects. This makes the encoding *much* smaller and easier to write.

→ Most planners translate the schematic input into (propositional) STRIPS in a pre-process, by instantiating the variables in all possible ways. This is called **grounding**.

Schematic Actions: Example

The schematic action:

$$\begin{aligned}x &\in \{car1, car2\} \\ y_1 &\in \{SB, KL\}, \\ y_2 &\in \{SB, KL\}, y_1 \neq y_2 \\ &(\{at(x, y_1)\}, \{at(x, y_2)\}, \{at(x, y_1)\})\end{aligned}$$

corresponds to the actions:

$$\begin{aligned}&(\{at(car1, SB)\}, \{at(car1, KL)\}, \{at(car1, SB)\}), \\&(\{at(car1, KL)\}, \{at(car1, SB)\}, \{at(car1, KL)\}), \\&(\{at(car2, SB)\}, \{at(car2, KL)\}, \{at(car2, SB)\}), \\&(\{at(car2, KL)\}, \{at(car2, SB)\}, \{at(car2, KL)\})\end{aligned}$$

Schematic Actions: Quantification

Example

$\exists x \in \{A, B, C\} : at(x, SB)$ is a short-hand for?
 $at(A, SB) \vee at(B, SB) \vee at(C, SB).$

Quantification in Formulas

Finite disjunctions $\varphi(o_1) \vee \dots \vee \varphi(o_n)$ represented as

$\exists x \in \{o_1, \dots, o_n\} : \varphi(x).$

Finite conjunctions $\varphi(o_1) \wedge \dots \wedge \varphi(o_n)$ represented as

$\forall x \in \{o_1, \dots, o_n\} : \varphi(x).$

Quantification over Effects

Finite list of conditional effects WHEN $\varphi(o_i)$ DO $\psi(o_i)$ represented as

$\forall x \in \{o_1, \dots, o_n\} : \text{WHEN } \varphi(o_i) \text{ DO } \psi(o_i).$

Questionnaire

Question!

Is the grounding process polynomial in the size of its input?

(A): Yes

(B): No

→ If an action schema has k parameters, and there are n objects each of these parameters can be instantiated with, then there are n^k grounded actions. Same for predicates. **Grounding is exponential in operator and predicate arity.**

- In practice, this is often Ok, many domains have maximum arity 2 or 3.
- However, this is NOT always so! (E.g., natural language generation
→ **Next Chapter**)
- Grounding typically leads to more efficient planning in the cases where it is feasible; in the other cases, **lifted planning** is needed.
- There has been little research on lifted planning in the last 2 decades.
→ We are currently taking up such research in FAI: Instruction planning in Minecraft!

PDDL Basics

The Planning Domain Definition Language (PDDL):

- Variants used by almost all implemented planning systems.
- Supports a formalism comparable to what we have outlined above (including schematic operators and quantification).
- Syntax inspired by the Lisp programming language: e.g., prefix notation for formulas

```
(and (or (on A B) (on A C))  
      (or (on B A) (on B C))  
      (or (on C A) (on A B)))
```
- The planner input is separated into a **domain file** (predicates, types, action schemas) and a **problem file** (objects, initial state, goal).

→ Some examples to get your feet wet:

<http://fai.cs.uni-saarland.de/hoffmann/PlanningForDummies.zip>

PDDL Domain Files

A PDDL domain file consists of:

1. (define (domain <name>)
2. A [requirements](#) definition (use “:adl :typing” by default).
3. Definitions of [types](#) (each object variable has a type).
4. Definitions of [predicates](#).
5. Definitions of [action schemas](#).

PDDL Problem Files

A PDDL problem file consists of:

1. `(define (problem <name>)`
2. `(:domain <name>)`
 - to which domain does this problem belong?
3. Definitions of objects belonging to each type.
4. Definition of the initial state (list of **ground predicates** initially true).
5. Definition of the goal (a formula like action preconditions).

PDDL Exercises

1. Box
2. Tower of Hanoi
3. BlocksWorld
4. Logistics
5. Miconic

Exercises Shared Folder: bit.ly/3N5EyWa

Online PDDL Editor: planning.domains

Exercise

- (a) Axiomatize the following scenario, appropriately with action precondition and effect axioms, and obtain successor state axioms.

Fluents:

- `boxClosed(s)` - The box is closed in situation `s`.
- `robotCloseToBox(s)` - The robot is close to the box in situation `s`.
- `objectInsideBox(s)` - The object is inside the box in situation `s`.

Actions:

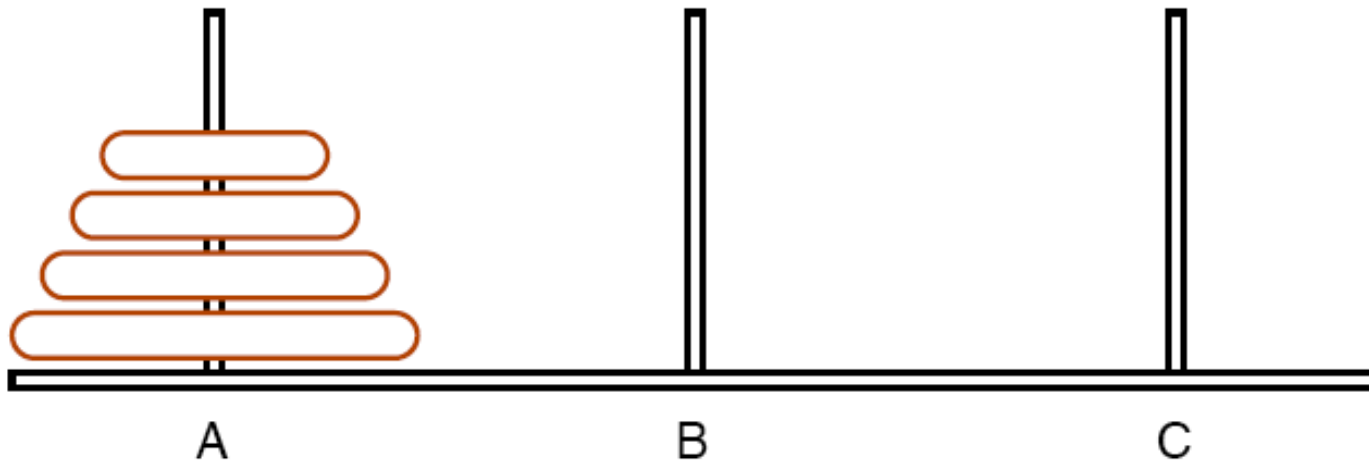
- `moveCloseToBox` - The robot moves close to the box. This can be done if the robot is not close to the box, and has the effect that the robot will be close to the box.
- `openBox` - The robot opens the box. This can be done if the robot is close to the box and the box is closed, and has the effect that the box will be open (that is, not closed).
- `extractObject` - The robot extracts the object inside the box. This requires that the box is open and the robot is close to the box, and has the effect that the object will be outside the box.

Initial situation description: Initially the robot is not close to the box, the box is closed, and the object is inside the box.

- (b) Show, by applying regression, that the object will be outside the box after the sequence of actions `moveCloseToBox`, `openBox`, `extractObject`, and that the sequence of actions is indeed executable.

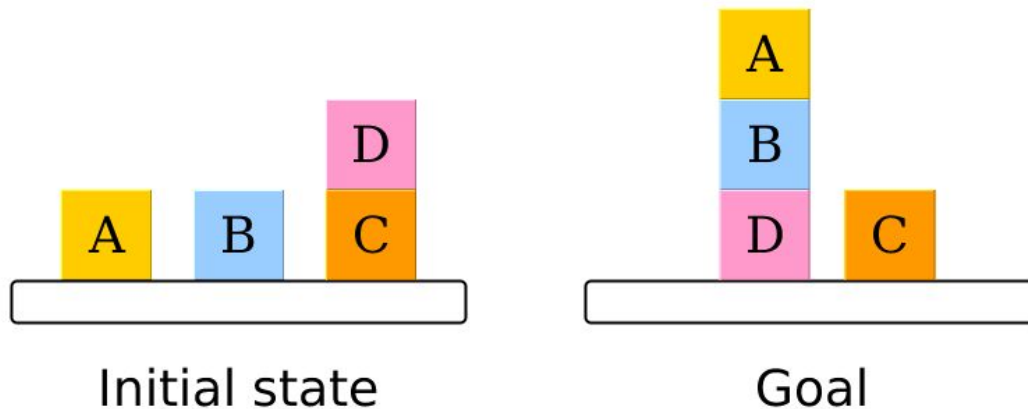
The Tower of Hanoi

- There are 3 pegs (A,B,C) and a tower of disks on the first one (A), with the smaller on the top and the bigger on the bottom. The purpose of the puzzle is to move the whole tower from the first peg (A) to the third (C), by moving only one disk every time and by observing not to put a bigger disk atop of a smaller one.



BlocksWorld

- Given a set of blocks sitting on a table, the goal is to build one or more vertical stacks of blocks. Only one block can be moved at a time: it may either be placed on the table or placed on top of another block. Because of this, any blocks that are, at a given time, under another block cannot be moved.



Example - Initial Situation

We consider a domain in which we have:

- 1 truck T
- 3 packages $P1, P2, P3$
- 4 locations $L1, L2, L3, L4$ connected among themselves in a certain way (see below)

Initially the truck and all the packages are in location $L1$. Moreover, nothing is on the truck.

The situation can be represented as the following DB:

at

object	location
T	L1
P1	L1
P2	L2
P3	L3

on

package	truck

connected

location	location
L1	L2
L2	L3
L3	L4
L2	L4
L1	L4
L4	L1

In FOL:

$$\begin{aligned} at(o, \ell, S_0) &\equiv //fuent \\ &(o = T \wedge \ell = L1) \vee (o = P1 \wedge \ell = L1) \vee (o = P2 \wedge \ell = L1) \vee (o = P3 \wedge \ell = L1) \\ on(p, t, S_0) &\equiv //fuent \\ &false \\ location(\ell_1, \ell_2) &\equiv //non-fuent predicate \\ &(\ell_1 = L1 \wedge \ell_2 = L2) \vee (\ell_1 = L2 \wedge \ell_2 = L3) \vee (\ell_1 = L3 \wedge \ell_2 = L4) \vee \\ &(\ell_1 = L2 \wedge \ell_2 = L4) \vee (\ell_1 = L1 \wedge \ell_2 = L2) \vee (\ell_1 = L4 \wedge \ell_2 = L1) \end{aligned}$$

Example - Actions

We consider 3 action each with its preconditions and effects (sort of postconditions):

- $drive(t, \ell_1, \ell_2)$
 - ▶ **Pre:** $at(t, \ell_1, s) \wedge connected(\ell_1, \ell_2)$
 - ▶ **Effects:** $at(t, \ell_2, s') \wedge \neg at(t, \ell_1, s')$ where $s' = do(drive(t, \ell_1, \ell_2), s)$
- $load(p, t)$
 - ▶ **Pre:** $\exists \ell. at(p, \ell, s) \wedge at(t, \ell, s) \wedge \neg \exists t'. on(p, t', s)$
 - ▶ **Effects:** $\forall \ell. \neg at(p, \ell, s') \wedge on(p, t, s')$ where $s' = do(load(p, t), s)$
- $unload(t)$
 - ▶ **Pre:** $\exists p. on(p, t, s)$
 - ▶ **Effects:** $\forall p. on(p, t, s) \rightarrow ((\forall \ell. at(t, \ell, s) \rightarrow at(p, \ell, s')) \wedge \neg on(p, t, s'))$ where $s' = do(unload(t), s)$

Example - Preconditions and Effects

Precondition axioms:

$$Poss(drive(t, \ell_1, \ell_2), s) \equiv \\ at(t, \ell_1, s) \wedge connected(\ell_1, \ell_2)$$

$$Poss(load(p, t), s) \equiv \\ \exists \ell. at(p, \ell, s) \wedge at(t, \ell, s) \wedge \neg \exists t'. on(p, t', s)$$

$$Poss(unload(t), s) \equiv \\ \exists p. on(p, t, s)$$

Effect axioms:

$$a = drive(t, \ell_1, \ell_2) \rightarrow at(t, \ell_2, do(a, s)) \\ a = drive(t, \ell_1, \ell_2) \rightarrow \neg at(t, \ell_1, do(a, s))$$

$$a = load(p, t) \rightarrow \neg at(p, \ell, do(a, s)) \\ \text{NB: equivalent to } a = load(p, t) \rightarrow \forall \ell. \neg at(p, \ell, do(a, s)) \\ a = load(p, t) \rightarrow on(p, t, do(a, s))$$

$$a = unload(t) \wedge on(p, t, s) \wedge at(t, \ell, s) \rightarrow at(p, \ell, do(a, s)) \\ \text{NB: equivalent to } a = unload(t) \rightarrow \forall p. (on(p, t, s) \rightarrow (\forall \ell. at(t, \ell, s) \rightarrow at(p, \ell, do(a, s)))) \\ a = unload(t) \wedge on(p, t, s) \rightarrow \neg on(p, t, do(a, s)) \\ \text{NB: equivalent to } a = unload(t) \rightarrow \forall p. (on(p, t, s) \rightarrow \neg on(p, t, do(a, s)))$$

The Miconic Elevator

- A number of passengers should be transported between different floors of a building. The elevator can go up or down between floors and at each floor passengers can enter or leave the elevator. For each passenger we are given an origin and a destination floor. We are also given the initial floor the elevator is on.



Planners

- Given a PDDL domain, a PDDL problem, a Planner finds a plan if exists

$$\textit{Problem in PDDL} \implies \boxed{\textit{Planner}} \implies \textit{Plan}$$

- Planning is computationally *intractable*, but planners can typically solve problems fast
- Planners implement efficient search algorithms (e.g., A*, IDA*, ...)

Main Properties:

- Generality: solve arbitrary problem instances
- Efficiency/Scalability: use effective heuristics

Planners (cont'd)

- All available Planners: <https://planning.wiki/ref/planners/atoz>
- Fast Forward (Hoffmann) 2000: <https://fai.cs.uni-saarland.de/hoffmann/ff.html>
- Fast Downward (Helmert) 2004:
 - <https://www.fast-downward.org/>
 - GitHub: <https://github.com/aibasel/downward>
 - Available heuristics: <https://www.fast-downward.org/Doc/Evaluator>
- [International Planning Competition](#) (IPC)
- At the latest IPC, most planners are built upon FD

Planning.domains Plugins

[Planning.domains](#) supports several plugins that help analyzing planning instances

Among all available plugins:

- PDDL problem generator
- Statespace Visualization
- [Planimation](#)
- Heuristic Visualization

Animation for Hanoi and BlocksWorld

PDDL History

The development of PDDL is mainly driven by the **International Planning Competition (IPC)**:

- **1998:** [PDDL](#) [McDermott *et al.* (1998)]
STRIPS and ADL.
- **2000:** [“PDDL subset for the 2000 competition”](#) [Bacchus (2000)]
STRIPS and ADL.
- **2002:** [PDDL2.1, Levels 1-3](#) [Fox and Long (2003)]
Numeric and temporal planning.
- **2004:** [PDDL2.2](#) [Hoffmann and Edelkamp (2005)]
Derived predicates and timed initial literals.
- **2006:** [PDDL3](#) [Gerevini *et al.* (2009)]
Soft goals and trajectory constraints.

PDDL in 2004

PDDL2.1 was (and is still today) considered a challenge, so Stefan Edelkamp and I made only two relatively minor language extensions for **PDDL2.2**:

- **Derived predicates**: Predicates that are not affected by the actions. Their value is instead derived via a set of derivation rules of the form **IF** $\varphi(\bar{x})$ **THEN** $P(\bar{x})$.
Example: Flow of current in an electricity network.

```
(:derived (fed ?x)  
          (exists ?y (and (connected ?x ?y) (fed ?y))))
```
- **Timed Initial Literals**: Literals that will become true, independently of the actions taken, at a pre-specified point in time.
Example: Opening/closing times.

```
(at 9 (shop-open)) (at 18 (not (shop-open)))
```


Summary

- PDDL is the de-facto standard for classical planning, as well as extensions to numeric/temporal planning, soft goals, trajectory constraints.
- PDDL is used in the [International Planning Competition \(IPC\)](#).
- PDDL uses a [schematic encoding](#), with [variables ranging over objects](#) similarly as in predicate logic. Most implemented systems use [grounding](#) to transform this into a propositional encoding.
- PDDL has a [Lisp-like syntax](#).